

Homework Assignment 5

This document contains the write-ups for the three challenges of **Assignment 5**.

Problem ID	Lab Name	Captured Flag	Steps
Problem 1	Xperience	<code>fsuCTF{p1ca55o_w0u1dv3_l0v3d_m5pa1n7}</code>	- Analyze Disk Image: Decompress the <code>.7z</code> file and identify the Virtual Hard Disk (<code>.vhd</code>) as a Windows XP installation. - Locate User Data: Mount the disk and navigate to <code>C:\Documents and Settings\daniel</code> to find a <code>README.rtf</code> mentioning a "masterpiece". - Trace Recent Activity: Check the <code>Recent</code> folder for shortcuts, identifying <code>masterpiece.bmp.lnk</code> which pointed to a file in the <code>Application Data</code> folder. - Extract Flag: Open the recovered <code>masterpiece.bmp</code> image with a viewer to reveal the flag.
Problem 2	Jaws	<code>fsuCTF{Y0ur3_G0nn4_N33d_A_Bi6g3r_8o47}</code>	- Traffic Analysis: Open the <code>jaws.pcap</code> file in Wireshark and filter for suspicious HTTP GET requests to the IP address <code>178.63.67.153</code>. - Identify Encoding: Notice a repetitive UUID-like path with a <code>?q=</code> parameter containing Base64-encoded strings. - Reconstruct Data: Manually extract and concatenate the Base64 values from the query parameters in chronological order. - Decode Flag: Use CyberChef to decode the full Base64 string to reveal the flag.
Problem 3	Whats the title again?	<code>fsuCTF{1_d0n7_w4n7_t0_pl4y_4nym0r3}</code>	- Process Identification: Use Volatility on the memory dump (<code>whats_the_title_again.raw</code>) to identify active processes like <code>wmplayer.exe</code>. - Inspect Command Line: Check <code>windows.cmdline</code> to see the media player's initial file, though the default sample song was a distraction. - Locate the song title written in leetspeak within the memory strings

PROBLEM 1 - Xperience

This challenge provides us with a file named `xp_smol.7z`.

1. Analysis

We started by decompressing the file using `7z x xp_smol.7z`, which gave us `xp_smol.vhd`. Running the `file` command confirmed this was a **Virtual Hard Disk**.

```
└─$ file xp_smol.vhd
xp_smol.vhd: Microsoft Disk Image, Virtual Server or Virtual PC, Creator vbox 7.2 (W2k) Tue Feb 10 01:59:15 2026, 42
94967296 bytes, CHS 8322/16/63
```

This meant it was likely to be a file system, so I mounted the disk on a Linux machine to explore the directories.

To mount it, I first needed to identify which partition contained the data, which I determined using `virt-filesystems -a xp_smol.vhd`. The output showed that the data was located in `sda1`.

```
mkdir /mnt/xp_disk
sudo guestmount -a xp_smol.vhd -m /dev/sda1 --ro /mnt/xp_disk
```

I mounted it in **read-only mode** (`--ro`) to avoid altering the evidence.

After mounting the drive and listing its content, I immediately realized I was looking at a **Windows XP** installation.

```
└─$ ls -la
total 295257
drwxrwxrwx 1 root root      4096 Feb 10 01:33 .
drwxr-xr-x 4 root root      4096 Feb 18 23:05 ..
-rwxrwxrwx 1 root root         0 Feb 10 01:19 AUTOEXEC.BAT
-rwxrwxrwx 1 root root       211 Feb 10 01:17 boot.ini
-rwxrwxrwx 1 root root         0 Feb 10 01:19 CONFIG.SYS
drwxrwxrwx 1 root root      4096 Feb 10 01:22 Documents and Settings
-rwxrwxrwx 1 root root         0 Feb 10 01:19 IO.SYS
-rwxrwxrwx 1 root root         0 Feb 10 01:19 MSDOS.SYS
-rwxrwxrwx 1 root root      47564 Apr 14 2008 NTDETECT.COM
-rwxrwxrwx 1 root root    250048 Apr 14 2008 ntldr
-rwxrwxrwx 1 root root 301989888 Feb 10 03:40 pagefile.sys
drwxrwxrwx 1 root root      4096 Feb 10 01:22 Program Files
drwxrwxrwx 1 root root         0 Feb 10 01:33 RECYCLER
drwxrwxrwx 1 root root      4096 Feb 10 01:21 System Volume Information
drwxrwxrwx 1 root root    28672 Feb 10 01:25 WINDOWS
```

In XP, user data isn't in `C:\Users`, but in `C:\Documents and Settings`. I identified a user named **daniel**.

Inside Daniel's Desktop, I found an `openssl.zip` containing a full OpenSSL Git repository. After searching the code and Git history for hidden clues, I discovered it was just a distraction and not related to the actual solution.

I decided to stop analyzing the source code and instead look through the rest of Daniel's files to see what else was there.

The first interesting thing I could find was a file called `README.rtf`, located in Daniel's **"My Documents"** folder. Inside, I found a note from Daniel explaining the challenge's lore:

```
└─$ cat README.rtf
{\rtf1\ansi\ansicpg1252\deff0\deflang1033{\fonttbl{\f0\fswiss\fcharset0 Arial;}}
{\*generator Msftedit 5.41.15.1515;}\viewkind4\uc1\pard\f0\fs20 Welcome, CTF'ers to my Windows XP machine. I love X
P because it's so simple, it provides an excellent place to make art and enjoy the wonderful digital environment whi
ch is 'bliss'. The only downside is that I was distracted when saving my masterpiece and now I don't remember where
I saved it :(\par
}
```

Since Daniel didn't remember where he saved his masterpiece, I decided to check his activity.

On Windows XP, the `Recent` folder stores `.lnk` (shortcut) files for every document or application the user has opened. I navigated to `/mnt/xp_disk/Documents and Settings/daniel/Recent` and found a shortcut named `masterpiece.bmp.lnk`.

```
ls -la
total 41
drwxrwxrwx 1 root root 4096 Feb 10 03:42 .
drwxrwxrwx 1 root root 4096 Feb 10 03:38 ..
-rwxrwxrwx 2 root root 679 Feb 10 02:17 abstract.bmp.lnk
-rwxrwxrwx 2 root root 391 Feb 10 01:24 boot.ini.lnk
-rwxrwxrwx 2 root root 672 Feb 10 02:17 bubbles.JPG.lnk
-rwxrwxrwx 1 root root 150 Feb 10 01:22 Desktop.ini
-rwxrwxrwx 2 root root 694 Feb 10 01:33 Desktop.ini.lnk
-rwxrwxrwx 2 root root 674 Feb 10 02:49 HACKING.md.lnk
-rwxrwxrwx 2 root root 293 Feb 10 01:24 'Local Disk (C:).lnk'
-rwxrwxrwx 2 root root 501 Feb 10 03:26 masterpiece.bmp.lnk
-rwxrwxrwx 2 root root 571 Feb 10 02:34 mosaic.GIF.lnk
-rwxrwxrwx 2 root root 448 Feb 10 02:34 'My Pictures.lnk'
-rwxrwxrwx 1 root root 457 Feb 10 02:49 openssl.lnk
-rwxrwxrwx 2 root root 397 Feb 10 02:45 openssl.zip.lnk
-rwxrwxrwx 2 root root 679 Feb 10 02:21 original.jpg.lnk
-rwxrwxrwx 1 root root 797 Feb 10 01:57 Paint.lnk
-rwxrwxrwx 2 root root 790 Feb 10 03:42 README.rtf.lnk
```

2. Solution Strategy

A `.lnk` file is more than a shortcut; it contains the absolute path to the target file. Even if the file is moved or "lost," the shortcut reveals where it was intended to be. I used `strings` on the shortcut and saw it pointed to

`C:\Documents and Settings\daniel\Desktop\masterpiece.bmp`, but the file wasn't there.

```
strings masterpiece.bmp.lnk
J:\C'
MASTER~1.BMP
J\XCJ\XC
C:\Documents and Settings\daniel\Desktop\masterpiece.bmp
xprienc-5bdfde
```

3. Execution and Flag

Since the "masterpiece" wasn't on the desktop, I assumed it was elsewhere, so I used the `find` command to scan the entire disk:

```
find /mnt/xp_disk/ -name "masterpiece.bmp"
/mnt/xp_disk/Documents and Settings/daniel/Application Data/Microsoft/Paint/masterpiece.bmp
```

Apparently if a program like Paint crashes or is closed unexpectedly, it can save recovery data or temporary files in the `Application Data` folder.

I first tried using `strings` on the BMP file, but found nothing. After some research, I learned that BMP is a Windows image format that stores pictures as a grid of pixels, so I simply opened it with an image viewer and that revealed the flag.

*\$uCTF{p1ca55
o_w0u1dv3_l0v3d_
m5pa1n7}

Flag: `fsuCTF{p1ca55o_w0u1dv3_l0v3d_m5pa1n7}`

PROBLEM 2 - Jaws

The challenge provides a single file named `jaws.pcap`. The prompt states that a network has been infiltrated and the traffic was captured to find evidence. The goal is to analyze the packet capture, identify the malicious activity, and recover the flag.

1. Analysis

Since I was given a `.pcap` file, I knew right away that this was a network traffic capture. To see what was happening inside the network, I opened the file using **Wireshark**.

I began with a preliminary visual inspection, scrolling through the packets to see which protocols were most frequent. I immediately noticed a mix of **TCP, DNS, and HTTP**.

- I first checked the **DNS** queries to see if there were any "weird" domains, but it just looked like standard queries.
- I then looked at the **HTTP** traffic. I noticed that among the regular traffic, there was a repetitive pattern of requests that didn't look like normal browsing.

1136	220.7367...	10.136.60...	52.149.246...	HTTP	99	GET / HTTP/1.1
1145	223.2457...	10.136.60...	178.63.67...	HTTP	140	GET /1a7b92af-e58d-41c2-9493-edcdb4066a8c?q=X0c= HTTP/1.1
1150	227.7233...	10.136.60...	46.105.204...	HTTP	97	GET / HTTP/1.1
1161	234.0736...	10.136.60...	146.201.11...	HTTP	96	GET / HTTP/1.1
1177	238.5096...	10.136.60...	13.249.98...	HTTP	94	GET / HTTP/1.1
1184	238.6803...	10.136.60...	44.199.238...	HTTP	99	GET / HTTP/1.1
1189	238.7923...	10.136.60...	15.197.167...	HTTP	108	GET / HTTP/1.1
1194	238.9098...	10.136.60...	13.249.98...	HTTP	94	GET / HTTP/1.1
1211	239.3093...	10.136.60...	142.250.64...	HTTP	100	GET / HTTP/1.1
1216	243.5066...	10.136.60...	140.82.114...	HTTP	95	GET / HTTP/1.1
1239	250.5227...	10.136.60...	172.64.155...	HTTP	96	GET / HTTP/1.1
1250	250.8163...	10.136.60...	23.0.162.1...	HTTP	97	GET / HTTP/1.1
1265	253.1733...	10.136.60...	198.49.23...	HTTP	109	GET / HTTP/1.1
1270	253.3047...	10.136.60...	52.149.246...	HTTP	99	GET / HTTP/1.1
1281	255.6453...	10.136.60...	52.23.127...	HTTP	107	GET / HTTP/1.1
1286	256.0014...	10.136.60...	178.63.67...	HTTP	140	GET /1a7b92af-e58d-41c2-9493-edcdb4066a8c?q=MG4= HTTP/1.1
1297	256.5638...	10.136.60...	146.201.11...	HTTP	96	GET / HTTP/1.1
1304	256.7236...	10.136.60...	104.21.96.2	HTTP	96	GET / HTTP/1.1
1317	259.0363...	10.136.60...	23.0.162.1...	HTTP	97	GET / HTTP/1.1
1328	259.2817...	10.136.60...	13.249.98...	HTTP	94	GET / HTTP/1.1
1335	259.4393...	10.136.60...	172.66.0.2...	HTTP	94	GET / HTTP/1.1
1340	259.5613...	10.136.60...	104.21.96.2	HTTP	96	GET / HTTP/1.1
1359	262.1087...	10.136.60...	23.0.162.1...	HTTP	97	GET / HTTP/1.1
1370	266.4533...	10.136.60...	15.197.167...	HTTP	108	GET / HTTP/1.1
1419	275.7514...	10.136.60...	52.149.246...	HTTP	99	GET / HTTP/1.1
1430	276.0203...	10.136.60...	46.105.204...	HTTP	97	GET / HTTP/1.1
1439	278.4403...	10.136.60...	178.63.67...	HTTP	140	GET /1a7b92af-e58d-41c2-9493-edcdb4066a8c?q=bjQ= HTTP/1.1
1450	280.9918...	10.136.60...	15.197.167...	HTTP	108	GET / HTTP/1.1

Every few packets, a specific internal host was reaching out to the same external destination. I saw that all these suspicious requests were directed to the IP address **178.63.67.153**. I applied a filter for that specific address: `http && ip.addr == 178.63.67.153`.

Once filtered, I could see that the traffic consisted of dozens of GET requests to a very long, unique UUID-like path: `/1a7b92af-e58d-41c2-9493-edcdb4066a8c`. Every request ended with a different value for a parameter named `q`.

190	35.558705	10.136.60...	178.63.67...	HTTP	140	GET /1a7b92af-e58d-41c2-9493-edcdb4066a8c?q=ZnM= HTTP/1.1
395	73.831871	10.136.60...	178.63.67...	HTTP	140	GET /1a7b92af-e58d-41c2-9493-edcdb4066a8c?q=dUM= HTTP/1.1
539	108.5566...	10.136.60...	178.63.67...	HTTP	140	GET /1a7b92af-e58d-41c2-9493-edcdb4066a8c?q=VEY= HTTP/1.1
666	138.7493...	10.136.60...	178.63.67...	HTTP	140	GET /1a7b92af-e58d-41c2-9493-edcdb4066a8c?q=e1k= HTTP/1.1
851	164.2618...	10.136.60...	178.63.67...	HTTP	140	GET /1a7b92af-e58d-41c2-9493-edcdb4066a8c?q=MHU= HTTP/1.1
977	185.9673...	10.136.60...	178.63.67...	HTTP	140	GET /1a7b92af-e58d-41c2-9493-edcdb4066a8c?q=cjM= HTTP/1.1
1145	223.2457...	10.136.60...	178.63.67...	HTTP	140	GET /1a7b92af-e58d-41c2-9493-edcdb4066a8c?q=X0c= HTTP/1.1
1286	256.0014...	10.136.60...	178.63.67...	HTTP	140	GET /1a7b92af-e58d-41c2-9493-edcdb4066a8c?q=MG4= HTTP/1.1
1439	278.4403...	10.136.60...	178.63.67...	HTTP	140	GET /1a7b92af-e58d-41c2-9493-edcdb4066a8c?q=bjQ= HTTP/1.1
1684	352.0634...	10.136.60...	178.63.67...	HTTP	140	GET /1a7b92af-e58d-41c2-9493-edcdb4066a8c?q=X04= HTTP/1.1
1791	379.5167...	10.136.60...	178.63.67...	HTTP	140	GET /1a7b92af-e58d-41c2-9493-edcdb4066a8c?q=MzM= HTTP/1.1
1948	419.4546...	10.136.60...	178.63.67...	HTTP	140	GET /1a7b92af-e58d-41c2-9493-edcdb4066a8c?q=ZF8= HTTP/1.1
2082	451.9074...	10.136.60...	178.63.67...	HTTP	140	GET /1a7b92af-e58d-41c2-9493-edcdb4066a8c?q=QV8= HTTP/1.1
2213	487.9494...	10.136.60...	178.63.67...	HTTP	140	GET /1a7b92af-e58d-41c2-9493-edcdb4066a8c?q=Qmk= HTTP/1.1
2324	521.6527...	10.136.60...	178.63.67...	HTTP	140	GET /1a7b92af-e58d-41c2-9493-edcdb4066a8c?q=Nmc= HTTP/1.1
2469	558.1997...	10.136.60...	178.63.67...	HTTP	140	GET /1a7b92af-e58d-41c2-9493-edcdb4066a8c?q=M3I= HTTP/1.1
2582	591.9927...	10.136.60...	178.63.67...	HTTP	140	GET /1a7b92af-e58d-41c2-9493-edcdb4066a8c?q=Xzg= HTTP/1.1
2790	638.4887...	10.136.60...	178.63.67...	HTTP	140	GET /1a7b92af-e58d-41c2-9493-edcdb4066a8c?q=bzQ= HTTP/1.1
2974	671.9807...	10.136.60...	178.63.67...	HTTP	140	GET /1a7b92af-e58d-41c2-9493-edcdb4066a8c?q=N30= HTTP/1.1

2. Solution Strategy

I focused on the `?q=` parameter. I noticed that the values ended with an `=` sign. This is a very common indicator of **Base64 encoding**.

To see if I was on the right track, I decided to manually test the first few values using an online decoder:

1. `ZnM=` → `fs`
2. `dUM=` → `uC`
3. `VEY=` → `TF`

When I put those together, I got `fsuCTF`. It was clear that the flag was being sent piece by piece through these query parameters.

The plan now was to:

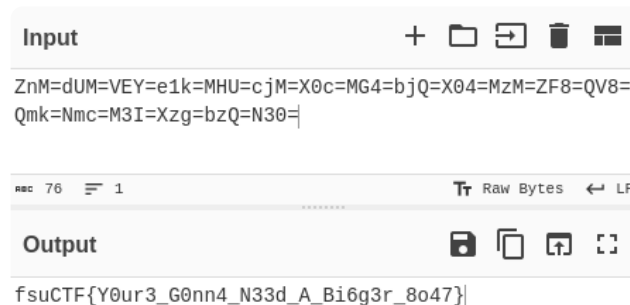
1. Extract all the strings from the `q=` parameter in chronological order.
2. Concatenate them into one long string.
3. Decode the final string from Base64 to reveal the flag.

3. Execution and Flag

Since the number of packets wasn't excessive, I manually extracted the values of the `q` parameter and concatenated them into a single string:

```
ZnM=dUM=VEY=e1k=MHU=cjM=X0c=MG4=bjQ=X04=MzM=ZF8=QV8=Qmk=Nmc=M3I=Xzg=bzQ=N30=
```

I then pasted the reconstructed string into CyberChef, decoded it, and obtained the flag.



Flag: `fsuCTF{Y0ur3_G0nn4_N33d_A_Bi6g3r_8o47}`

PROBLEM 3 - Whats the title again?

The goal of this challenge is to recover the title of a song that a user was listening to based on a provided memory dump. The instructions mentioned that the flag is not wrapped in the standard `fsuCTF{}` format inside the memory, so we must identify the string and wrap it ourselves.

The provided files for this challenge were:

- `memory.vmem`
- `what.raw`
- `whats_the_title_again.raw`

Although three memory images were provided, I chose to focus my analysis on `whats_the_title_again.raw`, as its filename directly matched the title of the challenge.

1. Analysis

I began by seeing what was happening on the machine at the time of the dump so I listed the active processes.

```
vol -f whats_the_title_again.raw windows.pslist
```

```
l- $ vol -f whats_the_title_again.raw windows.pslist
```

Volatility 3 Framework 2.27.0 Write-Ups
Progress: 100.00 PDB scanning finished

PID	PPID	ImageFileName	Offset(V)	Threads	Handles	SessionId	Wow64	CreateTime	ExitTime	File output
4	0	System	0x84236b38	66	372	N/A	False	2026-02-10 19:51:26.000000 UTC	N/A	Disabled
212	4	smss.exe	0x850f9d40	2	29	N/A	False	2026-02-10 19:51:26.000000 UTC	N/A	Disabled
280	272	csrss.exe	0x851e24f8	9	368	0	False	2026-02-10 19:51:27.000000 UTC	N/A	Disabled
316	272	wininit.exe	0x842cf4d0	3	76	0	False	2026-02-10 19:51:32.000000 UTC	N/A	Disabled
328	308	csrss.exe	0x842d0100	7	242	1	False	2026-02-10 19:51:32.000000 UTC	N/A	Disabled
356	308	winlogon.exe	0x85365d40	6	130	1	False	2026-02-10 19:51:32.000000 UTC	N/A	Disabled
416	316	services.exe	0x853d2030	9	191	0	False	2026-02-10 19:51:34.000000 UTC	N/A	Disabled
432	316	lsass.exe	0x853f96b8	7	472	0	False	2026-02-10 19:51:34.000000 UTC	N/A	Disabled
440	316	lsass.exe	0x853fb818	10	144	0	False	2026-02-10 19:51:34.000000 UTC	N/A	Disabled
544	416	svchost.exe	0x8543da58	11	364	0	False	2026-02-10 19:51:35.000000 UTC	N/A	Disabled
612	416	svchost.exe	0x854569a8	6	255	0	False	2026-02-10 19:51:35.000000 UTC	N/A	Disabled
836	416	sppsvc.exe	0x84ba5950	5	150	0	False	2026-02-10 16:51:40.000000 UTC	N/A	Disabled
876	416	svchost.exe	0x84bb0d40	13	284	0	False	2026-02-10 16:51:41.000000 UTC	N/A	Disabled
900	416	svchost.exe	0x84bc73c0	37	955	0	False	2026-02-10 16:51:41.000000 UTC	N/A	Disabled
948	416	svchost.exe	0x84bd2988	21	425	0	False	2026-02-10 16:51:41.000000 UTC	N/A	Disabled
980	948	audiodg.exe	0x84bd7478	9	143	0	False	2026-02-10 16:51:41.000000 UTC	N/A	Disabled
1016	416	svchost.exe	0x84be39d0	11	223	0	False	2026-02-10 16:51:43.000000 UTC	N/A	Disabled
1040	416	svchost.exe	0x85453030	15	323	0	False	2026-02-10 16:51:43.000000 UTC	N/A	Disabled
1140	416	TrustedInstall	0x84c03520	4	115	0	False	2026-02-10 16:51:44.000000 UTC	N/A	Disabled
1308	416	spoolsv.exe	0x84c36d40	13	263	0	False	2026-02-10 16:51:45.000000 UTC	N/A	Disabled
1340	416	svchost.exe	0x84d452e8	19	299	0	False	2026-02-10 16:51:45.000000 UTC	N/A	Disabled
1752	416	SearchIndexer.	0x84e1f030	15	672	0	False	2026-02-10 16:51:51.000000 UTC	N/A	Disabled
1896	1752	SearchProtocol	0x84ec6030	9	496	0	False	2026-02-10 16:51:54.000000 UTC	N/A	Disabled
100	416	taskhost.exe	0x84d5e8a8	9	180	1	False	2026-02-10 16:51:55.000000 UTC	N/A	Disabled
236	876	dmw.exe	0x84e10d40	4	69	1	False	2026-02-10 16:51:55.000000 UTC	N/A	Disabled
492	312	explorer.exe	0x84e22030	46	980	1	False	2026-02-10 16:51:55.000000 UTC	N/A	Disabled
1224	1212	rundll32.exe	0x84f24030	4	67	1	False	2026-02-10 16:51:56.000000 UTC	N/A	Disabled
1424	1396	rundll32.exe	0x84f31408	4	66	1	False	2026-02-10 16:51:56.000000 UTC	N/A	Disabled
564	492	regsvr32.exe	0x84d4a4e8	0	-	1	False	2026-02-10 16:51:57.000000 UTC	2026-02-10 16:51:58.000000 UTC	Disabled
584	416	svchost.exe	0x84e1b030	13	206	0	False	2026-02-10 16:53:46.000000 UTC	N/A	Disabled
1700	416	svchost.exe	0x84de3d40	10	297	0	False	2026-02-10 16:53:47.000000 UTC	N/A	Disabled
1552	544	WmiPrvSE.exe	0x84e2d1c8	7	126	0	False	2026-02-10 16:54:34.000000 UTC	N/A	Disabled
2004	1872	wmplayer.exe	0x8438e030	18	477	1	False	2026-02-10 16:54:39.000000 UTC	N/A	Disabled
1872	416	taskhost.exe	0x843973f8	10	159	0	False	2026-02-10 16:54:45.000000 UTC	N/A	Disabled
2116	492	MineSweeper.ex	0x843c66d8	9	201	1	False	2026-02-10 16:54:48.000000 UTC	N/A	Disabled
2184	1752	SearchFilterHo	0x84397830	6	163	0	False	2026-02-10 16:54:54.000000 UTC	N/A	Disabled
2276	492	notepad.exe	0x84ef9348	1	56	1	False	2026-02-10 16:54:56.000000 UTC	N/A	Disabled

I spotted two processes that seemed likely to correspond to programs being used by the user.

- `wmplayer.exe` (PID 2004): The Windows Media Player.
- `notepad.exe` (PID 2276): A text editor.

I decided to see what was in the Notepad. To analyze the process (PID 2276), I dumped its memory using Volatility:

```
vol -f whats_the_title_again.raw windows.memmap --pid 2276 --dump
```

I then ran `strings` on it to extract readable strings from the dump and search for references to the term "flag".

```
strings -e l pid.2276.dmp | grep -i "flag"
```

```

-$ strings -e l pid.2276.dmp | grep -i "flag"
windows_tracing_flags=3
flags=3
8suCTF{the_flag_is_not_wrapped_k
?TF{the_flag_is_not_wrapped_
*suCTF{the_flag_is_not_wrapped_keep_
fsuCTF{the_flag_is_not_wrapped_keep_looking!}
dwFlags
dwFlags
Allow flag to be passed with CreateFile call that indicates to perform downgrade if applicable.
\Registry\Machine\Software\Microsoft\Windows NT\CurrentVersion\AppCompatFlags
Flags

```

The output showed several lines that confirmed I was looking in the right places, but it told me that the actual flag was hidden elsewhere and I shouldn't expect it to look like a flag.

I shifted the focus to understanding how the processes were being used. I started by checking the command line arguments, as this could reveal file paths or specific tracks being opened by a media player.

```
vol -f whats_the_title_again.raw windows.cmdline
```

The output for `notepad.exe` showed nothing interesting but the output for `wmplayer.exe` showed it had been launched with the file `Sleep Away.mp3`.

```

1700 svchost.exe C:\Windows\System32\svchost.exe -k secsvcs
1552 WmiPrvSE.exe C:\Windows\system32\wbem\wmiprvse.exe
2004 wmplayer.exe "C:\Program Files\Windows Media Player\wmplayer.exe" /Relaunch /Play "C:\Users\Public\Music\Sample Music\Sleep Away.mp3"
1872 taskhost.exe taskhost.exe $(Arg0)
2116 Minesweeper.exe "C:\Program Files\Microsoft Games\minesweeper\minesweeper.exe"
2184 SearchFilterHost.exe "C:\Windows\system32\SearchFilterHost.exe" 0 508 512 520 65536 516
2276 notepad.exe "C:\Windows\system32\notepad.exe"

```

At first I thought this could be the song that the user was listening to but the flag `fsuCTF{Sleep Away.mp3}` wasn't accepted as a solution. After some research I learned that **"Sleep Away"** by Bob Acri is a famous default sample song included with Windows 7.

I suspected the player might have started with this song, but the user likely changed it later. I needed a way to see what the Media Player was currently showing to the user.

Typically, a media player's song title appears in the window's title bar, so I proceeded to attempt its recovery.

2. Execution and Flag

I tried to use the `windows.windows.Windows` plugin to list window titles, but I hit a technical wall: the plugin failed, stating it only supports x64 versions of Windows.

I decided to search the entire memory dump for the string "Windows Media Player", as window titles for this app usually follow the pattern: `[Song Title] - [Artist] - Windows Media Player`.

```

-$ strings -e l whats_the_title_again.raw | grep "\- Windows Media Player"
1_d0n7_w4n7_t0_pl4y_4nym0r3 - Bob Acri - Windows Media Player
1_d0n7_w4n7_t0_pl4y_4nym0r3 - Bob Acri - Windows Media Player

```

This time, the song name appeared in leetspeak, which confirmed that we had successfully identified the flag.

Flag: fsuCTF{1_d0n7_w4n7_t0_pl4y_4nym0r3}